

std::constexpr_v

Document #: P2781R1
Date: 2023-01-28
Project: Programming Language C++
Audience: LEWG-I
LEWG
Reply-to: Matthias Kretz
<m.kretz@gsi.de>
Zach Laine
<whatwasthataddress@gmail.com>

Contents

- 1 Changelog
 - 1.1 Changes since R0
- 2 Relationship to previous work
- 3 The ergonomics of std::integral_constant<int> are bad
 - 3.1 Replacing the uses of std::integral_constant is not enough
 - 3.2 The difference in template parameters to std::constexpr_v and std::c_
- 4 Making constexpr_v more useful
- 5 What about strings?
- 6 An example using operator()
- 7 A concept for detecting types like std::constexpr_v
- 8 Design
 - 8.1 Add constexpr_v
 - 8.2 Add constexpr_value
 - 8.3 Add a feature macro
- 9 Implementation experience
- 10 Possible polls for LEWG
- 11 Wording

§ 1 Changelog

§ 1.1 Changes since R0

- Remove discussion of literals for std::integral_constant and std::constexpr_v, based on LEWG feedback.
- Change the title to reflect the loss of the literals.
- Add an explicit example involving std::constexpr_v::operator(), as requested by LEWG.
- Add concept std::constexpr_value, as suggested by LEWG.
- Wording.

§ 2 Relationship to previous work

This paper is co-authored by the authors of P2725R1 (“std::integral_constant Literals”) and P2772R0 (“std::integral_constant literals do not suffice — constexpr_v?”). This paper supersedes both of those previous papers.

§ 3 The ergonomics of std::integral_constant<int> are bad

std::integral_constant<int> is used in lots of places to communicate a constant integral value to a given interface. The length of its spelling makes it very verbose. Fortunately, we can do a lot better.

Before	After
<pre>// From P2630R1 auto const sir = std::strided_index_range{std::integral_constant<size_t, 0>{}, std::integral_constant<size_t, 10> {}, 3}; auto y = submdspan(x, sir);</pre>	<pre>auto y = submdspan(x, std::strided_index_range{ std::c_<0>, std::c_<10>, 3});</pre>

The “after” case above would require that std::strided_index_range be changed; that is not being proposed here. The point of the example is to show the relative convenience of std::integral_constant versus the proposed std::constexpr_v.

§ 3.1 Replacing the uses of std::integral_constant is not enough

Parameters passed to a constexpr function lose their constexpr-ness when used inside the function. Replacing std::integral_constant with std::constexpr_v has the potential to improve a lot more uses of compile-time constants than just integrals; what about all the other constexpr-friendly C++ types?

Consider:

```
template<typename T>
struct my_complex
{
    T re, im;
};

inline constexpr short foo = 2;

template<typename T>
struct X
{
    void f(auto c)
    {
        // c is to be used as a constexpr value here
    }
};
```

We would like to be able to call x.f() with a value, and have that value keep its constexpr-ness. Let’s introduce a template “constexpr_v” that holds a constexpr value that it is given as an non-type template parameter.

```
namespace std {
    template<class T, T X>
        requires same_as<remove_cvref_t<T>, T>
        struct constexpr_v
        {
```

```

static constexpr T value = X;

using value_type = T;
using type = constexpr_v;

constexpr operator value_type() const { return X; }

// The rest of the members are discussed below ....
};
}

```

Now we can write this.

```

template<typename T>
void g(X<T> x)
{
    x.f(std::constexpr_v<int, 1>{});
    x.f(std::constexpr_v<std::size_t, 2uz>{});
    x.f(std::constexpr_v<double, 3.0>{});
    x.f(std::constexpr_v<float, 4.f>{});
    x.f(std::constexpr_v<short, foo>{});
    x.f(std::constexpr_v<my_complex, my_complex(1.f, 1.f)>{});
}

```

Let's now add a constexpr variable template with a shorter name, say c_.

```

namespace std {
    template<auto X>
    inline constexpr constexpr_v<remove_const_t<decltype(X)>, X> c_{};
}

```

And now we can write this.

```

template<typename T>
void g(X<T> x)
{
    x.f(std::c_<1>);
    x.f(std::c_<2uz>);
    x.f(std::c_<3.0>);
    x.f(std::c_<4.f>);
    x.f(std::c_<foo>);
    x.f(std::c_<my_complex(1.f, 1.f)>);
}

```

§ 3.2 The difference in template parameters to std::constexpr_v and std::c_

std::c_ takes an auto NTTP. std::constexpr_v takes a type T and an NTTP x for type T. Why is this?

The rationale is that when a user writes std::c_<foo>, we want the notation to be as terse as possible. Moreover, the user is in complete control of the x value that she puts in the brackets. In a generic context, that might not always be the case; you sometimes might want to force the type held by your std::constexpr_v to be a particular type, say size_t. For instance, say you have an existing function foo():

```

template<size_t X>
auto foo(std::constexpr_v<size_t, X> x) {
    return /* ... */;
}

```

And now let's say you want to write a function bar() such that bar<X>() calls foo(std::constexpr_v<size_t, X> {}):

```

template<auto X>
void bar() {
    std::constexpr_v<size_t, X> x;
}

```

```

    auto y = bar(x);
    // Use y ....
}

```

If you didn't care what type the `T` parameter to `std::constexpr_v` was, you could use `std::c_<X>` instead. Having both notational options is useful.

§ 4 Making `constexpr_v` more useful

`constexpr_v` is essentially a wrapper. It takes a value `x` of some structural type `T`, and represents `x` in such a way that we can continue to use `x` as a compile-time constant, regardless of context. As such, `constexpr_v` should be implicitly convertible to `T`; this is already reflected in the design presented above. For the same reason, `constexpr_v` should provide all the operations that the underlying type has. Though we cannot predict what named members the underlying type `T` has, we *can* guess at all the operator overloads it might have.

So, by adding conditionally-defined overloads for all the overloadable operators, we can make `constexpr_v` as natural to use as many of the types it might wrap.

```

namespace std {
    template<class T, T X>
        requires same_as<remove_cvref_t<T>, T>
    struct constexpr_v
    {
        static constexpr T value = X;

        using value_type = T;
        using type = constexpr_v;

        constexpr operator value_type() const { return X; }

        // unary -
        constexpr auto operator-() const
            requires requires { constexpr_v<decltype(-X), -X>{}; }
            { return constexpr_v<decltype(-X), -X>{}; }

        // binary + and -
        template<class U>
            friend constexpr constexpr_v<decltype(X + U::value), (X + U::value)>
                operator+(constexpr_v, U) { return {}; }
        template<class U>
            requires not_constexpr_v<U>
            friend constexpr constexpr_v<decltype(U::value + X), (U::value + X)>
                operator+(U, constexpr_v) { return {}; }
        template<class U>
            friend constexpr constexpr_v<decltype(X - U::value), (X - U::value)>
                operator-(constexpr_v, U) { return {}; }
        template<class U>
            requires not_constexpr_v<U>
            friend constexpr constexpr_v<decltype(U::value - X), (U::value - X)>
                operator-(U, constexpr_v) { return {}; }

        // etc... (full listing later)
    };
}

```

These operators are defined in such a way that they behave just like the operations on underlying the `T` and `U` values would, including promotions and coercions. For example:

```

static_assert(std::is_same_v<
    decltype(std::c_<42> - std::c_<13u>),
    std::constexpr_v<unsigned int, 29u>>);

```

Each operation is only defined if the underlying operation on x and y is defined. Each operation additionally requires that the result of the underlying operation have a structural type.

The mutating operations are left out, because none of them makes sense – the type of the mutated value would have to change, since the value is itself part of the type.

All the remaining operations are included, even the index and call operators. The rationale for this is that a user may want to make some sort of compile-time domain-specific embedded language using operator overloading, and having all but a couple of the operators specified would frustrate that effort.

The only downside to adding `std::constexpr_v::operator()()` is that it would represent a break from the design of `std::integral_constant`, making it an imperfect drop-in replacement for that template.

The operators are designed to interoperate with other types and templates that have a `constexpr` static value member. This works with `std::constexpr_vs` of course, but also `std::integral_constants`, and user-provided types as well. For example:

```
struct my_type { constexpr static int value = 42; };

void foo()
{
    constexpr auto zero = my_type{} - std::c_<42>; // Ok.
    // ...
}
```

Note that the addition of these operators is in line with the poll:

“Add a new robust integral constant type with all the numerical operators, as proposed in P2772R0, and use that for these literals instead of `std::integral_constant`”?

SF	F	N	A	SA
4	7	1	1	1

... taken in the 2023-01-17 Library Evolution telecon.

Note that the one SA said he would not be opposed if the word “integral” was stricken from the poll, and the design of `std::constexpr_v` is not limited to integral types.

§ 5 What about strings?

As pointed out on the reflector, `std::c_<"foo">` does not work, because of language rules. However, it’s pretty easy for users to add an NTTP-friendly string wrapper type, and then use that with `std::c_<>`.

```
template<size_t N>
struct strlit
{
    constexpr strlit(char const (&str)[N]) { std::copy_n(str, N, value); }

    template<size_t M>
    constexpr bool operator==(strlit<M> rhs) const
    {
        return std::ranges::equal(bytes_, rhs.bytes_);
    }

    friend std::ostream & operator<<(std::ostream & os, strlit l)
    {
        assert(!l.value[N - 1] && "value must be null-terminated");
        return os.write(l.value, N - 1);
    }

    char value[N];
};
```

```
};

int main()
{
    auto f = std::c_strlit("foo");
    std::cout << f; // Prints "foo".
}
```

§ 6 An example using operator ()

The addition of non-arithmetic operators may seem academic at first. However, consider this constexpr-friendly parser combinator mini-library.

```
namespace parse {

    template<typename L, typename R>
    struct or_parser;

    template<size_t N>
    struct str_parser
    {
        template<size_t M>
        constexpr bool operator()(strlit<M> lit) const
        {
            return lit == str_;
        }
        template<typename P>
        constexpr auto operator|(P parser) const
        {
            return or_parser<str_parser, P>{*this, parser};
        }
        strlit<N> str_;
    };

    template<typename L, typename R>
    struct or_parser
    {
        template<size_t M>
        constexpr bool operator()(strlit<M> lit) const
        {
            return l_(lit) || r_(lit);
        }
        template<typename P>
        constexpr auto operator|(P parser) const
        {
            return or_parser<or_parser, P>{*this, parser};
        }
        L l_;
        R r_;
    };

}

int foo()
{
    constexpr parse::str_parser p1{strlit("neg")};
    constexpr parse::str_parser p2{strlit("incr")};
    constexpr parse::str_parser p3{strlit("decr")};

    constexpr auto p = p1 | p2 | p3;

    constexpr bool matches_empty = p(strlit(""));
    static_assert(!matches_empty);
    constexpr bool matches_pos = p(strlit("pos"));
    static_assert(!matches_pos);
    constexpr bool matches_decr = p(strlit("decr"));
}
```

```
static_assert(matches_decr);
}
```

(This relies on the `strlit` struct shown just previously.)

Say we wanted to use the templates in namespace `parser` along side other values, like `ints` and `floats`. We would want that not to break our `std::constexpr_v` expressions. Having to work around the absence of `std::constexpr_v::operator()` would require us to write a lot more code. Here is the equivalent of the function `foo()` above, but with all the variables wrapped using `std::c_`.

```
int bar()
{
    constexpr parse::str_parser p1{strlit("neg")};
    constexpr parse::str_parser p2{strlit("incr")};
    constexpr parse::str_parser p3{strlit("decr")};

    constexpr auto p_ = std::c_<p1> | std::c_<p2> | std::c_<p3>;

    constexpr bool matches_empty_ = p_(std::c_<strlit("")>);
    static_assert(!matches_empty_);
    constexpr bool matches_pos_ = p_(std::c_<strlit("pos")>);
    static_assert(!matches_pos_);
    constexpr bool matches_decr_ = p_(std::c_<strlit("decr")>);
    static_assert(matches_decr_);
}
```

As you can see, everything works as it did before. The presence of `operator()` does not enable any new functionality, it just keeps code that happens to use it from breaking.

§ 7 A concept for detecting types like `std::constexpr_v`

It was suggested in the 2023-02 Issaquah meeting review that it might be useful also to standardize a concept that could be used to detect `std::constexpr_v`-like types – that is, types that have a static, `constexpr` data member called `value`. Such a concept is easy to write.

```
template<typename T, typename ValueType>
concept constexpr_value = requires { constexpr ValueType x = T::value; };
```

This concept allows you to conveniently express the requirements on function parameters that must be usable as constant expressions within your function.

```
template<typename T, T Value>
struct my_constant
{
    static constexpr T value = Value;

    // Other API ...
};

auto plus(constexpr_value<int> auto x, constexpr_value<int> auto y)
{ return std::c_<x.value + y.value>; }

void h()
{
    // All results are equal to std::c_<1>.
    auto result_1 = plus(std::c_<0>, std::c_<1>); // Ok.
    auto result_2 = plus(std::c_<0>, std::integral_constant<int, 1>{}); // Also ok.
    auto result_3 = plus(std::c_<0>, my_constant<int, 1>{}); // Still ok.
}
```

However, you sometimes might want to constrain a parameter to types that contain a `value` member of any type. For example, say you want to define `std::constexpr_v`-style operator overloads for several constant-wrapping

types you might have, without relying on `std::constexpr_v`'s operators at all. For that, you would want a slightly different concept.

```
template<typename T>
concept constexpr_value = requires { constexpr auto x = T::value; };

auto operator+(constexpr_value auto x, constexpr_value auto y)
    -> decltype(std::c_<decltype(x)::value + decltype(y)::value>)
{ return std::c_<x.value + y.value>; }

void i()
{
    // result is equal to std::c_<1>.
    auto result = std::integral_constant<int, 0>{} + my_constant<int, 1>{};
}
```

In the design below, the concept is called `constexpr_value`, and it is possible to use it in both the ways just demonstrated.

§ 8 Design

§ 8.1 Add `constexpr_v`

```
namespace std {
    template<class T, T X>
        requires same_as<remove_cvref_t<T>, T>
        struct constexpr_v;

    template<class T>
        constexpr bool not-constexpr-t = true; // exposition only
    template<class T, T X>
        constexpr bool not-constexpr-t<constexpr_v<T, X>> = false; // exposition only

    template<class T, T X>
        requires same_as<remove_cvref_t<T>, T>
        struct constexpr_v
        {
            static constexpr T value = X;

            using value_type = T;
            using type = constexpr_v;

            constexpr operator value_type() const { return X; }

            constexpr auto operator+() const
                requires requires { constexpr_v<decltype(+X), +X>{}; }
            { return constexpr_v<decltype(+X), +X>{}; }
            constexpr auto operator-() const
                requires requires { constexpr_v<decltype(-X), -X>{}; }
            { return constexpr_v<decltype(-X), -X>{}; }
            constexpr auto operator~() const
                requires requires { constexpr_v<decltype(~X), ~X>{}; }
            { return constexpr_v<decltype(~X), ~X>{}; }
            constexpr auto operator!() const
                requires requires { constexpr_v<decltype(!X), !X>{}; }
            { return constexpr_v<decltype(!X), !X>{}; }
            constexpr auto operator&() const
                requires requires { constexpr_v<decltype(&X), &X>{}; }
            { return constexpr_v<decltype(&X), &X>{}; }
            constexpr auto operator*() const
                requires requires { constexpr_v<decltype(*X), *X>{}; }
            { return constexpr_v<decltype(*X), *X>{}; }

            template<class U>
                friend constexpr constexpr_v<decltype(X << U::value), (X << U::value)>
```



```

    operator<<(constexpr_v, U) { return {}; }
template<class U>
    requires not-constexpr-t<U>
    friend constexpr constexpr_v<decltype(U::value << X), (U::value << X)>
        operator<<(U, constexpr_v) { return {}; }
template<class U>
    friend constexpr constexpr_v<decltype(X >> U::value), (X >> U::value)>
    operator>>(constexpr_v, U) { return {}; }
template<class U>
    requires not-constexpr-t<U>
    friend constexpr constexpr_v<decltype(U::value >> X), (U::value >> X)>
    operator>>(U, constexpr_v) { return {}; }
template<class U>
    friend constexpr constexpr_v<decltype(X * U::value), (X * U::value)>
    operator*(constexpr_v, U) { return {}; }
template<class U>
    requires not-constexpr-t<U>
    friend constexpr constexpr_v<decltype(U::value * X), (U::value * X)>
    operator*(U, constexpr_v) { return {}; }
template<class U>
    friend constexpr constexpr_v<decltype(X / U::value), (X / U::value)>
    operator/(constexpr_v, U) { return {}; }
template<class U>
    requires not-constexpr-t<U>
    friend constexpr constexpr_v<decltype(U::value / X), (U::value / X)>
    operator/(U, constexpr_v) { return {}; }
template<class U>
    friend constexpr constexpr_v<decltype(X % U::value), (X % U::value)>
    operator%(constexpr_v, U) { return {}; }
template<class U>
    requires not-constexpr-t<U>
    friend constexpr constexpr_v<decltype(U::value % X), (U::value % X)>
    operator%(U, constexpr_v) { return {}; }
template<class U>
    friend constexpr constexpr_v<decltype(X + U::value), (X + U::value)>
    operator+(constexpr_v, U) { return {}; }
template<class U>
    requires not-constexpr-t<U>
    friend constexpr constexpr_v<decltype(U::value + X), (U::value + X)>
    operator+(U, constexpr_v) { return {}; }
template<class U>
    friend constexpr constexpr_v<decltype(X - U::value), (X - U::value)>
    operator-(constexpr_v, U) { return {}; }
template<class U>
    requires not-constexpr-t<U>
    friend constexpr constexpr_v<decltype(U::value - X), (U::value - X)>
    operator-(U, constexpr_v) { return {}; }

template<class U>
    friend constexpr constexpr_v<decltype(X < U::value), (X < U::value)>
    operator<(constexpr_v, U) { return {}; }
template<class U>
    requires not-constexpr-t<U>
    friend constexpr constexpr_v<decltype(U::value < X), (U::value < X)>
    operator<(U, constexpr_v) { return {}; }
template<class U>
    friend constexpr constexpr_v<decltype(X > U::value), (X > U::value)>
    operator>(constexpr_v, U) { return {}; }
template<class U>
    requires not-constexpr-t<U>
    friend constexpr constexpr_v<decltype(U::value > X), (U::value > X)>
    operator>(U, constexpr_v) { return {}; }
template<class U>
    friend constexpr constexpr_v<decltype(X <= U::value), (X <= U::value)>
    operator<=(constexpr_v, U) { return {}; }
template<class U>
    requires not-constexpr-t<U>

```

```

        friend constexpr constexpr_v<decltype(U::value <= X), (U::value <= X)>
            operator<=(U, constexpr_v) { return {}; }
template<class U>
    friend constexpr constexpr_v<decltype(X >= U::value), (X >= U::value)>
        operator>=(constexpr_v, U) { return {}; }
template<class U>
    requires not-constexpr-t<U>
        friend constexpr constexpr_v<decltype(U::value >= X), (U::value >= X)>
            operator>=(U, constexpr_v) { return {}; }
template<class U>
    friend constexpr constexpr_v<decltype(X == U::value), (X == U::value)>
        operator==(constexpr_v, U) { return {}; }
template<class U>
    requires not-constexpr-t<U>
        friend constexpr constexpr_v<decltype(U::value == X), (U::value == X)>
            operator==(U, constexpr_v) { return {}; }
template<class U>
    friend constexpr constexpr_v<decltype(X != U::value), (X != U::value)>
        operator!=(constexpr_v, U) { return {}; }
template<class U>
    requires not-constexpr-t<U>
        friend constexpr constexpr_v<decltype(U::value != X), (U::value != X)>
            operator!=(U, constexpr_v) { return {}; }
template<class U>
    friend constexpr constexpr_v<decltype(X <=> U::value), (X <=> U::value)>
        operator<=>(constexpr_v, U) { return {}; }
template<class U>
    requires not-constexpr-t<U>
        friend constexpr constexpr_v<decltype(U::value <=> X), (U::value <=> X)>
            operator<=>(U, constexpr_v) { return {}; }

template<class U>
    friend constexpr constexpr_v<decltype(X && U::value), (X && U::value)>
        operator&&(constexpr_v, U) { return {}; }
template<class U>
    requires not-constexpr-t<U>
        friend constexpr constexpr_v<decltype(U::value && X), (U::value && X)>
            operator&&(U, constexpr_v) { return {}; }
template<class U>
    friend constexpr constexpr_v<decltype(X || U::value), (X || U::value)>
        operator||(constexpr_v, U) { return {}; }
template<class U>
    requires not-constexpr-t<U>
        friend constexpr constexpr_v<decltype(U::value || X), (U::value || X)>
            operator||(U, constexpr_v) { return {}; }
template<class U>
    friend constexpr constexpr_v<decltype(X & U::value), (X & U::value)>
        operator&(constexpr_v, U) { return {}; }
template<class U>
    requires not-constexpr-t<U>
        friend constexpr constexpr_v<decltype(U::value & X), (U::value & X)>
            operator&(U, constexpr_v) { return {}; }
template<class U>
    friend constexpr constexpr_v<decltype(X | U::value), (X | U::value)>
        operator|(constexpr_v, U) { return {}; }
template<class U>
    requires not-constexpr-t<U>
        friend constexpr constexpr_v<decltype(U::value | X), (U::value | X)>
            operator|(U, constexpr_v) { return {}; }
template<class U>
    friend constexpr constexpr_v<decltype(X ^ U::value), (X ^ U::value)>
        operator^(constexpr_v, U) { return {}; }
template<class U>
    requires not-constexpr-t<U>
        friend constexpr constexpr_v<decltype(U::value ^ X), (U::value ^ X)>
            operator^(U, constexpr_v) { return {}; }

```

```

template<class U>
    friend constexpr constexpr_v<decltype(X , U::value), (X , U::value)>
        operator,(constexpr_v, U) { return {}; }
template<class U>
    requires not-constexpr-t<U>
        friend constexpr constexpr_v<decltype(U::value , X), (U::value , X)>
            operator,(U, constexpr_v) { return {}; }
template<class U>
    friend constexpr constexpr_v<decltype(X ->* U::value), (X ->* U::value)>
        operator->*(constexpr_v, U) { return {}; }
template<class U>
    requires not-constexpr-t<U>
        friend constexpr constexpr_v<decltype(U::value ->* X), (U::value ->* X)>
            operator->*(U, constexpr_v) { return {}; }

template<class... Args>
    constexpr auto operator()(Args... args) const
        -> constexpr_v<decltype(X(Args::value...)), X(Args::value...)>
        { return {}; }
template<class... Args>
    constexpr auto operator()(Args... args) const
        -> constexpr_v<decltype(X[Args::value...]), X[Args::value...]>
        { return {}; }
};

template<auto X>
    inline constexpr constexpr_v<remove_const_t<decltype(X)>, X> c_{};
}

```

§ 8.2 Add constexpr_value

```

namespace std {
    template<typename T, typename ValueType = void>
        concept constexpr_value = see below;
}

```

This concept is equivalent to `requires { constexpr auto x = T::value; }` when `ValueType` is `void`, and `requires { constexpr ValueType x = T::value; }` otherwise.

§ 8.3 Add a feature macro

Add a new feature macro, `__cpp_lib_constexpr_v`.

§ 9 Implementation experience

Look up a few lines to see an implementation of `std::constexpr_v`. At the time of this writing, there is one caveat: `operator[]()` looks correct to the authors, but does not work in any compiler tested, due to the very limited multi-variate `operator[]` support in even the latest compilers.

Additionally, an `integral_constant` with most of the operator overloads has been a part of [Boost.Hana](#) since its initial release in May of 2016. Its operations have been used by many, many users.

§ 10 Possible polls for LEWG

— We should call `std::constexpr_v`:

- `std::constexpr_v`
- `std::constexpr_t`
- `std::constexpr_value`

- `std::constant_v`
- `std::constant_t`
- `std::constant_value`
- `std::const_v`
- `std::const_t`
- `std::const_value`
- We should call `std::c_`:
 - `std::c_`
 - `std::c`
 - `std::co`
 - `std::ct`
 - `std::_C`
 - `std::cx`
 - `std::c_v`
 - `std::cv`
 - `std::v_`
 - `std::_v`
 - `std::cn`
 - `std::con`
 - `std::cnt`
 - `std::cnst`
 - `std::const_`
 - `std::Const`
 - All of the above in a nested namespace.
- We want a concept like `std::constexpr_value` that identifies types that have a static, constexpr data member named `value` convertible to a specific type.
- We want a concept like `std::any_constexpr_value` that identifies types that have a static, constexpr data member named `value` of any type.

§ 11 Wording

Add the following to `[meta.type.synop]`, after `false_type`:

```
template<class T, T X>
    requires same_as<remove_cvref_t<T>, T>
struct constexpr_v;

template<class T>
    constexpr bool not-constexpr-t = true;           // exposition only
template<class T, T X>
    constexpr bool not-constexpr-t<constexpr_v<T, X>> = false; // exposition only

template<auto X>
    inline constexpr constexpr_v<remove_const_t<decltype(X)>, X> c_;
```

```
template<typename T, typename ValueType = void>
    concept constexpr_value = see below;
```

Add the following to [meta.help], after `integral_constant`:

```
template<class T, T X>
    requires same_as<remove_cvref_t<T>, T>
struct constexpr_v
{
    static constexpr T value = X;

    using value_type = T;
    using type = constexpr_v;

    constexpr operator value_type() const { return X; }

    constexpr auto operator+() const
        requires requires { constexpr_v<decltype(+X), +X>{}; }
        { return constexpr_v<decltype(+X), +X>{}; }
    constexpr auto operator-() const
        requires requires { constexpr_v<decltype(-X), -X>{}; }
        { return constexpr_v<decltype(-X), -X>{}; }
    constexpr auto operator~() const
        requires requires { constexpr_v<decltype(~X), ~X>{}; }
        { return constexpr_v<decltype(~X), ~X>{}; }
    constexpr auto operator!() const
        requires requires { constexpr_v<decltype(!X), !X>{}; }
        { return constexpr_v<decltype(!X), !X>{}; }
    constexpr auto operator&() const
        requires requires { constexpr_v<decltype(&X), &X>{}; }
        { return constexpr_v<decltype(&X), &X>{}; }
    constexpr auto operator*() const
        requires requires { constexpr_v<decltype(*X), *X>{}; }
        { return constexpr_v<decltype(*X), *X>{}; }

    template<class U>
        friend constexpr constexpr_v<decltype(X << U::value), (X << U::value)>
            operator<<(constexpr_v, U) { return {}; }
    template<class U>
        requires not-constexpr-t<U>
        friend constexpr constexpr_v<decltype(U::value << X), (U::value << X)>
            operator<<(U, constexpr_v) { return {}; }
    template<class U>
        friend constexpr constexpr_v<decltype(X >> U::value), (X >> U::value)>
            operator>>(constexpr_v, U) { return {}; }
    template<class U>
        requires not-constexpr-t<U>
        friend constexpr constexpr_v<decltype(U::value >> X), (U::value >> X)>
            operator>>(U, constexpr_v) { return {}; }
    template<class U>
        friend constexpr constexpr_v<decltype(X * U::value), (X * U::value)>
            operator*(constexpr_v, U) { return {}; }
    template<class U>
        requires not-constexpr-t<U>
        friend constexpr constexpr_v<decltype(U::value * X), (U::value * X)>
            operator*(U, constexpr_v) { return {}; }
    template<class U>
        friend constexpr constexpr_v<decltype(X / U::value), (X / U::value)>
            operator/(constexpr_v, U) { return {}; }
    template<class U>
        requires not-constexpr-t<U>
        friend constexpr constexpr_v<decltype(U::value / X), (U::value / X)>
            operator/(U, constexpr_v) { return {}; }
    template<class U>
        friend constexpr constexpr_v<decltype(X % U::value), (X % U::value)>
            operator%(constexpr_v, U) { return {}; }
    template<class U>
```

```

    requires not-constexpr-t<U>
    friend constexpr constexpr_v<decltype(U::value % X), (U::value % X)>
        operator%(U, constexpr_v) { return {}; }
template<class U>
    friend constexpr constexpr_v<decltype(X + U::value), (X + U::value)>
        operator+(constexpr_v, U) { return {}; }
template<class U>
    requires not-constexpr-t<U>
    friend constexpr constexpr_v<decltype(U::value + X), (U::value + X)>
        operator+(U, constexpr_v) { return {}; }
template<class U>
    friend constexpr constexpr_v<decltype(X - U::value), (X - U::value)>
        operator-(constexpr_v, U) { return {}; }
template<class U>
    requires not-constexpr-t<U>
    friend constexpr constexpr_v<decltype(U::value - X), (U::value - X)>
        operator-(U, constexpr_v) { return {}; }

template<class U>
    friend constexpr constexpr_v<decltype(X < U::value), (X < U::value)>
        operator<(constexpr_v, U) { return {}; }
template<class U>
    requires not-constexpr-t<U>
    friend constexpr constexpr_v<decltype(U::value < X), (U::value < X)>
        operator<(U, constexpr_v) { return {}; }
template<class U>
    friend constexpr constexpr_v<decltype(X > U::value), (X > U::value)>
        operator>(constexpr_v, U) { return {}; }
template<class U>
    requires not-constexpr-t<U>
    friend constexpr constexpr_v<decltype(U::value > X), (U::value > X)>
        operator>(U, constexpr_v) { return {}; }
template<class U>
    friend constexpr constexpr_v<decltype(X <= U::value), (X <= U::value)>
        operator<=(constexpr_v, U) { return {}; }
template<class U>
    requires not-constexpr-t<U>
    friend constexpr constexpr_v<decltype(U::value <= X), (U::value <= X)>
        operator<=(U, constexpr_v) { return {}; }
template<class U>
    friend constexpr constexpr_v<decltype(X >= U::value), (X >= U::value)>
        operator>=(constexpr_v, U) { return {}; }
template<class U>
    requires not-constexpr-t<U>
    friend constexpr constexpr_v<decltype(U::value >= X), (U::value >= X)>
        operator>=(U, constexpr_v) { return {}; }
template<class U>
    friend constexpr constexpr_v<decltype(X == U::value), (X == U::value)>
        operator==(constexpr_v, U) { return {}; }
template<class U>
    requires not-constexpr-t<U>
    friend constexpr constexpr_v<decltype(U::value == X), (U::value == X)>
        operator==(U, constexpr_v) { return {}; }
template<class U>
    friend constexpr constexpr_v<decltype(X != U::value), (X != U::value)>
        operator!=(constexpr_v, U) { return {}; }
template<class U>
    requires not-constexpr-t<U>
    friend constexpr constexpr_v<decltype(U::value != X), (U::value != X)>
        operator!=(U, constexpr_v) { return {}; }
template<class U>
    friend constexpr constexpr_v<decltype(X <=> U::value), (X <=> U::value)>
        operator<=>(constexpr_v, U) { return {}; }
template<class U>
    requires not-constexpr-t<U>
    friend constexpr constexpr_v<decltype(U::value <=> X), (U::value <=> X)>
        operator<=>(U, constexpr_v) { return {}; }

```

```

template<class U>
    friend constexpr constexpr_v<decltype(X && U::value), (X && U::value)>
        operator&&(constexpr_v, U) { return {}; }
template<class U>
    requires not-constexpr-t<U>
        friend constexpr constexpr_v<decltype(U::value && X), (U::value && X)>
            operator&&(U, constexpr_v) { return {}; }
template<class U>
    friend constexpr constexpr_v<decltype(X || U::value), (X || U::value)>
        operator||(constexpr_v, U) { return {}; }
template<class U>
    requires not-constexpr-t<U>
        friend constexpr constexpr_v<decltype(U::value || X), (U::value || X)>
            operator||(U, constexpr_v) { return {}; }
template<class U>
    friend constexpr constexpr_v<decltype(X & U::value), (X & U::value)>
        operator&(constexpr_v, U) { return {}; }
template<class U>
    requires not-constexpr-t<U>
        friend constexpr constexpr_v<decltype(U::value & X), (U::value & X)>
            operator&(U, constexpr_v) { return {}; }
template<class U>
    friend constexpr constexpr_v<decltype(X | U::value), (X | U::value)>
        operator|(constexpr_v, U) { return {}; }
template<class U>
    requires not-constexpr-t<U>
        friend constexpr constexpr_v<decltype(U::value | X), (U::value | X)>
            operator|(U, constexpr_v) { return {}; }
template<class U>
    friend constexpr constexpr_v<decltype(X ^ U::value), (X ^ U::value)>
        operator^(constexpr_v, U) { return {}; }
template<class U>
    requires not-constexpr-t<U>
        friend constexpr constexpr_v<decltype(U::value ^ X), (U::value ^ X)>
            operator^(U, constexpr_v) { return {}; }

template<class U>
    friend constexpr constexpr_v<decltype(X , U::value), (X , U::value)>
        operator,(constexpr_v, U) { return {}; }
template<class U>
    requires not-constexpr-t<U>
        friend constexpr constexpr_v<decltype(U::value , X), (U::value , X)>
            operator,(U, constexpr_v) { return {}; }
template<class U>
    friend constexpr constexpr_v<decltype(X ->* U::value), (X ->* U::value)>
        operator->*(constexpr_v, U) { return {}; }
template<class U>
    requires not-constexpr-t<U>
        friend constexpr constexpr_v<decltype(U::value ->* X), (U::value ->* X)>
            operator->*(U, constexpr_v) { return {}; }

template<class... Args>
    constexpr auto operator()(Args... args) const
        -> constexpr_v<decltype(X(Args::value...)), X(Args::value...)>
        { return {}; }
template<class... Args>
    constexpr auto operator()(Args... args) const
        -> constexpr_v<decltype(X[Args::value...]), X[Args::value...]>
        { return {}; }
};

template<auto X>
    inline constexpr constexpr_v<remove_const_t<decltype(X)>, X> c_{};

```

- 2 The class template `constexpr_v` aids in metaprogramming by ensuring that the evaluation of expressions comprised entirely of `constexpr_v`s are core constant expressions (`[expr.const]`), regardless of the context in

which they appear. In particular, this enables use of `constexpr_v` values that are passed as arguments to `constexpr` functions to be used as template parameters.

- 3 The variable template `c_` is provided as a convenient way to nominate `constexpr_v` values.

```
template<typename T, typename ValueType = void>
    concept constexpr_value = see below;
```

- 4 The `constexpr_value` concept specifies the requirements on types that are compatible with the operator overloads of `constexpr_v`. `constexpr_value` is equivalent to `requires { constexpr auto x = T::value; }` if `ValueType` is `void`, and `requires { constexpr ValueType x = T::value; }` otherwise.

Add to `[version.syn]`:

```
#define __cpp_lib_constexpr_v XXXXXL // also in <type_traits>
```